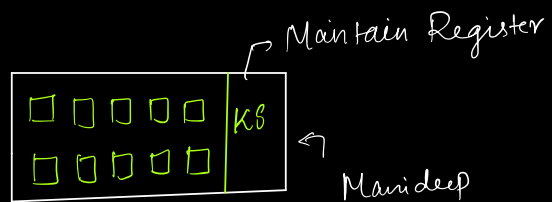


Hashing Introduction :-

Kshitiz Shrivastava → Hotel business



expanding
1000 rooms.

Program → based indexing → availability
 $\text{bool ch}[1001] = T$

Pandemic

$\text{ch}[10] = F$ (Not available)

Numerologist

↓

1000 lucky Numbers
in range $[1 - 10^9]$

$\text{bool ch}[10^9 + 1] = T$

⇒ Huge space loss

Adv:- Access time - $O(1)$

$[4, 7, 11, 19, 25, \dots]$

Keys have to be unique

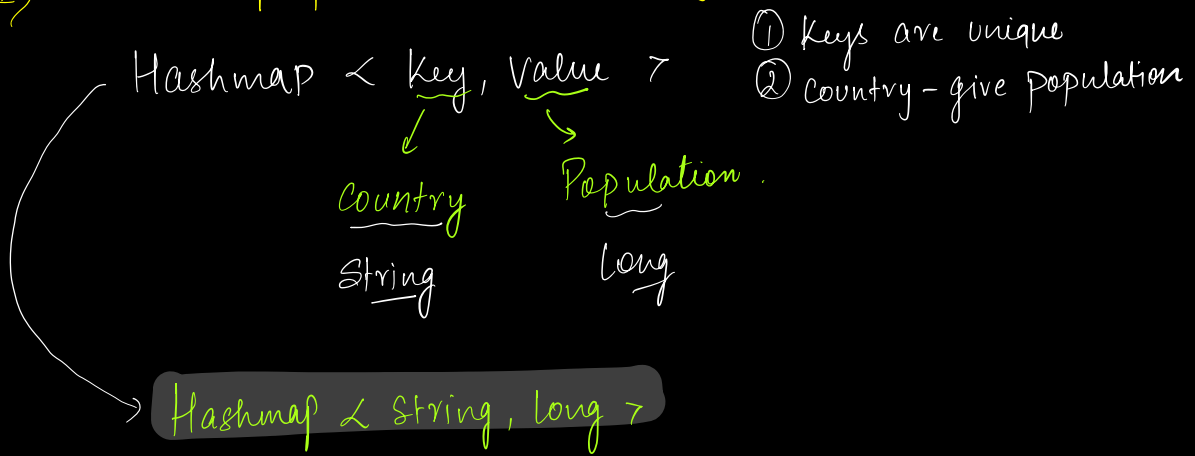
HashMap < Key, Value >

Hashset < Keys >

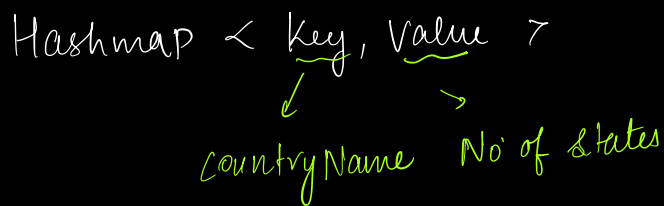
(If we want to store
only keys)

HM { < 79, occupied >
< 100, occupied >
< 134, occupied > }

Q1) Store population of every country.

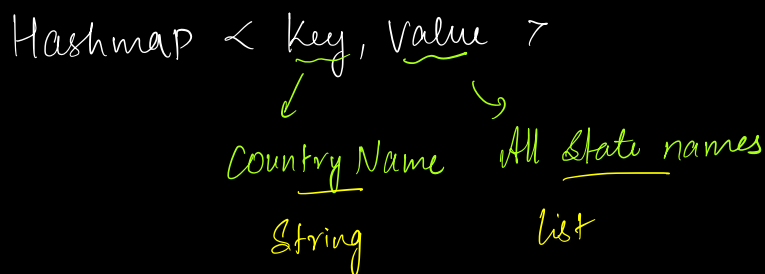


Q2) No of states in each country.



HashMap < String, int >

Q3) For every country we want to store all state names.



HashMap < String, List >

Q4) For every country, store population of each state.

HashMap < key, value >

countryName
↳ String

Population of every state
↳ HashMap < stateName, long >

HashMap < String, HashMap < String, long > >

// Value - Can be anything

// Keys - { int, long, float, String ... }

↳ As a key, can we keep a list



	<u>Java</u>	<u>C++</u>	<u>Python</u>	<u>JS</u>	<u>C</u>
HashMap	HashMap	unordered_map	Dictionary	map	{ }
Hashset	Hashset	unordered_set	Set	set	{ }

Terminology - HashMap < K, V > / Hashset < K >

→ You'll have to check syntax in your own language of choice

HashMap < Key, Value >

All above operations

⇒ $O(1)$ avg case

{
Insert < Key, value >
Search < key >
remove < key, value >
update < key, newval >
Size() → Number of keys
present in hashmap.
}

→ Keys have to be unique

HashSet < Keys >

{
Insert(key)
Search(key)
remove(key)
modify(keys) {
 → remove (old key)
 → insert (new key)
}
Size() → No of keys present
}

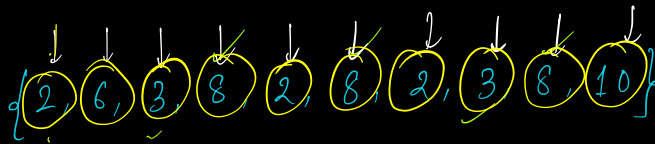
→ $O(1)$ avg case.

Q1) Find frequency of Numbers.

Given N array elements & Q queries. for each query find frequency of each element in array.

Occurrences

N = 10



Query = 4

2 : 3

8 : 3

3 : 2

5 : 0

HashMap < Key, Value >

array elements
int
frequency
int

HashMap < int, int > HM;

(check syntax before implementing)

HM

< 2, 3 >

< 6, 1 >

< 3, 2 >

< 8, 3 >

< 10, 1 >

Idea:- For every query, iterate & get count.

Tc: $O(Q * N)$

Sc: $O(1)$

Query = 4

2 : 3 → hm[2] : 3

8 : 3 → hm[8] : 3

3 : 2 → hm[3] : 2

5 : 0 → 0
(check & access)

// Pseudo Code

HashMap <int, int> hm;

for(int i=0; i < N; i++) {

if(ar[i] is in hm) { // search ar[i] in hm

 // update value by 1

 hm[ar[i]]++ // update value by 1

}

else {

 // insert <ar[i], 1> in hm

 hm.insert({ar[i], 1})

}

}

} $O(N)$

for(int i=0; i < Q; i++) {

 // x is query data

if(x is in hm) { // search - $O(1)$

 // access value of x from hm

 print(hm[x])

}

else {

 print(0)

}

}

} $O(Q)$

TC: $O(Q+N)$, SC: $O(N)$

2Q) Find the first non repeating element.

Eg arr[6] = {1, 2, 3, 1, 2, 5} } ans = 3
 x x ✓

Eg2 arr[8] = {4, 3, 3, 2, 5, 6, 4, 5} } ans = 2.
 x x x ✓

Eg3 arr[7] = {2, 6, 8, 4, 7, 2, 9} } ans = 6
 x ✓

Idea

- 1) Insert all elements in hm
- 2) Iterate in hashmap & get 1st key with freq = 1

Note:- In hashmap insertion order is not maintained, when we print hashmap we can get any order.

Idea 2:-

- 1) Insert all elements in hm
- 2) Iterate on array & get 1st element with freq = 1

```
for(int i=0; i<N; i++){
    if (hm[arr[i]] == 1){
        return arr[i]
    }
}
```

TC : $O(N+N) = O(N)$

SC : $O(N)$

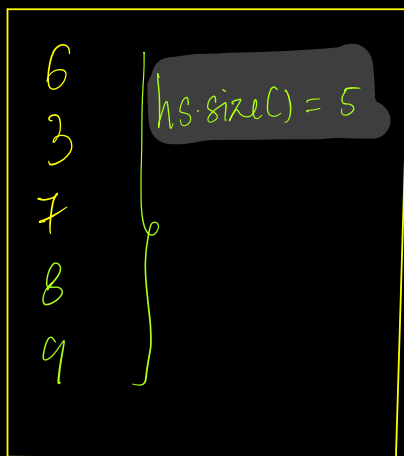
3Q) Given N array elements find no of distinct elements.
(Unique)

Eg1 $arr[5] = \{ \underset{+1}{3}, \underset{+1}{5}, \underset{+1}{6}, \underset{+1}{5}, 4 \}$ ans = 4

Eg2 $arr[7] = \{ \checkmark 6, \checkmark 3, \checkmark 7, \checkmark 3, \checkmark 8, \checkmark 6, \checkmark 9 \}$ ans = 5

HashSet <key> $\rightarrow$ array elements
 $\hookrightarrow$ int

HashSet <int> hs



Note: Even if we insert 2 same keys in HashSet, it will store 1 unique occurrence

HashSet <int> hs

for(int i=0; i<N; i++) {
 hs.insert(arr[i])

}

return hs.size()

TC: O(N), SC: O(N)

4Q) Given N array elements, check if all elements are distinct or not.

$\{3, 8, 8, 3, 5, 4, 7\} \rightarrow 5$ No
 $\{3, 8, 5, 4, 1, 6\} \rightarrow 6$ Yes

1) Insert all elements in hashset $hs.size() < N$
 $\hookrightarrow$ repetitions

2) if ($hs.size() == N$) {

 // all are distinct

}

else {

 there is a repetition

}

5Q) Given N array elements, check if there exists a boolean subarray with sum = 0

	0	1	2	3	4	5	6	7	8	9
arr[] :	2	2	1	-3	4	3	1	-2	-3	2
PF[] :	(2)	4	(5)	(2)	6	9	10	8	(5)	7

Idea:-

1) For every subarray
generate sum == 0

App1:- $O(N^3)$

App2:- $O(N^2)$

$$PF[2] = 5 \Rightarrow [0-2]$$

$$PF[8] = 5$$

$$\hookrightarrow [0-8] = [0-2] + [3-8]$$

$$5 = 5 + [3-8]$$

$$\text{Sum}[3-8] = 0$$

$$PF[0] = 2 \Rightarrow [0-0]$$

$$PF[3] = 2$$

$$\hookrightarrow [0-3] = [0-0] + [1-3]$$

$$\begin{array}{ccc} \downarrow & & \downarrow \\ 2 & & 2 \end{array} + [1-3]$$

$$\text{Sum}[1-3] = 0$$

Ex:- $arr[3]: 3 \ 0 \ 4 \}$ ✓

$PF[]: 3 \ 3 \ 7 \}$ repetition ✓ Subarray ✓

Ex:- $arr[n]: -1 \ 4 \ -3 \ 2 \}$
 $\swarrow$ Subarray with sum = 0

$PF[]: -1 \ 3 \ 0 \ 2 \}$ No Repetition, ✗ No subarray.

Edge Case

① Either on Sun/Mon

② Either in normal schedule